

Panduan Alur Kerja Sistem — Jawaban Teknikal untuk Sidang

Bank jawaban lisan untuk pertanyaan bertipe "**bagaimana sebenarnya cara kerja X di sistem kamu?**" — pertanyaan yang tidak bisa dijawab dengan menunjuk layar, harus keluar dari mulut.

Ini **bukan** peta codebase. Tidak ada path file atau nomor baris di sini; untuk itu pakai [Panduan Live Coding](#). Yang ada di sini: mekanisme sistem, dijelaskan setingkat orang yang membangunnya.

Cara memakai

Tiap entri punya tiga lapis:

Lapis	Fungsi	Perlakuan
Jawaban inti	15–20 detik. Cukup untuk sebagian besar penguji.	Hafal mati. Ini yang keluar refleks.
Amunisi	Kalau penguji menggali: "kenapa begitu?", "kalau gagal bagaimana?"	Pahami logikanya, tak perlu hafal kata per kata.
Jebakan	Kalimat yang jangan sampai keluar, atau kelemahan yang harus diakui duluan.	Baca ulang malam sebelum sidang.

Aturan tempur: **jawab inti dulu, lalu diam.** Jangan tumpahkan amunisi kalau tidak diminta — menjawab lebih panjang dari yang ditanya justru membuka pertanyaan baru yang tidak perlu.

Nama teknologi boleh disebut. Sanctum, queue, SHA-512, Spatie Permission — itu bukan pamer, itu bukti kamu tahu apa yang kamu pasang.

Bagian A — Hak Akses & Autentikasi

A1 "Bagaimana proses login di sistem kamu, dan bagaimana sistem mengingat siapa yang sedang login?"

JAWABAN INTI Hafal mati

User memasukkan identifier — bisa username, bisa email, sistem yang menentukan mana — lalu password diverifikasi dengan hash bcrypt, bukan dibandingkan sebagai teks biasa. Kalau cocok dan akunnya aktif, backend menerbitkan token Sanctum yang berlaku 8 jam. Token itu disimpan di session frontend dan dikirim sebagai Bearer token di setiap permintaan berikutnya. Jadi backend-nya sendiri stateless — tidak menyimpan sesi, tiap permintaan membuktikan dirinya sendiri lewat token.

AMUNISI

- **Kenapa "identifier", bukan "username"?** Ada satu layanan khusus yang memutuskan apakah masukan itu username atau email, lalu mencari user yang cocok. Bagi pengguna, satu kolom login lebih sederhana daripada disuruh memilih dulu.

- **Password tidak pernah tersimpan.** Yang disimpan hash bcrypt. Verifikasi dilakukan dengan menghitung ulang hash dari masukan, bukan mendekripsi yang tersimpan — bcrypt memang tidak bisa dibalik.
- **Akun nonaktif ditolak setelah password benar dicek,** dengan pesan berbeda ("Akun tidak aktif, hubungi admin"). Password salah dan akun nonaktif adalah dua kondisi berbeda dan pesannya memang sengaja dibedakan supaya pengguna tahu harus berbuat apa.
- **Satu akun, satu sesi aktif.** Saat login berhasil, semua token lama milik user itu dicabut. Konsekuensinya: login di perangkat kedua otomatis menendang perangkat pertama. Ini keputusan sadar — akun bendahara dan operator memegang data keuangan, sesi yang menggantung di komputer lain adalah risiko.
- **Token membawa daftar kemampuan.** Saat diterbitkan, token diisi seluruh nama permission milik user sebagai *abilities*. Bersama itu dikembalikan juga peran, cabang aktif, dan penanda `must_change_password` untuk memaksa ganti sandi pada login pertama.
- **Token kedaluwarsa 8 jam,** diatur lewat konfigurasi, bukan angka yang tertanam di kode. Token yang lewat masa berlaku dibersihkan otomatis saat login berikutnya.

JEBAKAN

- Jangan bilang "pakai session". Backend-nya **tidak** pakai session; yang pakai session adalah frontend, dan itu cuma tempat menyimpan token. Kalau tertukar, penguji akan menganggap kamu tidak paham beda stateful dan stateless.
- Jangan bilang "password dienkripsi". Enkripsi bisa dibalik, hash tidak. Kata yang benar: **di-hash**.
- Kalau ditanya "kenapa tidak pakai JWT?" — jawab jujur: Sanctum sudah bagian dari ekosistem Laravel, mendukung pencabutan token per baris di database, dan kebutuhan sistem ini tidak menuntut token yang bisa diverifikasi tanpa menyentuh database. JWT justru menyulitkan pencabutan segera.

A2 "Bagaimana sistem menentukan siapa boleh mengakses apa?"

JAWABAN INTI Hafal mati

Kuncinya satu konsep: **resource key**. Setiap hal yang bisa diamankan — halaman, tombol, endpoint API — punya identitas berupa string seperti `siswa.create`. Kode program tidak pernah menyebut nama permission secara langsung, hanya menyebut resource key. Pemetaan dari resource key ke permission disimpan di database, bukan di kode. Jadi mengubah siapa boleh apa cukup lewat halaman Manajemen RBAC, tanpa menyentuh kode dan tanpa deploy ulang.

AMUNISI

- **Tiga tabel yang bekerja sama:** tabel `permissions` menyimpan daftar izin (dikelola Spatie Permission); `page_permissions` memetakan resource key ke permission untuk pengamanan sisi tampilan; `permission_endpoints` memetakan resource key ke permission untuk pengamanan sisi API. Dua pemetaan terakhir sengaja dipisah dan berdiri sendiri.
- **Saat login,** frontend memanggil satu endpoint yang mengembalikan daftar resource key milik user, lalu menyimpannya di session. Sesudah itu pengecekan tampilan tidak menyentuh database sama sekali — cukup memeriksa apakah string-nya ada di daftar. Itu sebabnya menu dan tombol bisa dirender cepat tanpa query berulang.
- **Superadmin dilewatkan** lewat mekanisme gerbang global Laravel, sebelum pengecekan permission apa pun dijalankan. Praktisnya: superadmin tidak bisa mengunci dirinya sendiri.

- **Resource key bisa dinonaktifkan tanpa dihapus.** Ada penanda aktif; memaatikannya mencabut akses seketika, tapi riwayat pemetaannya tetap ada kalau suatu saat mau diaktifkan lagi.
- **Konvensi penamaan** berlapis: `siswa` untuk level halaman, `siswa.create` untuk level aksi. Jadi seseorang bisa diberi hak melihat data siswa tanpa hak menambah.

JEBAKAN

- Jangan bilang "permission-nya di-hardcode". Justru sebaliknya, dan itu nilai jual utama modul ini.
- Kalau ditanya "berapa permission yang ada?" — jangan sebut angka pasti kalau belum mengeceknya pagi itu. Jumlahnya bisa berubah karena permission memang bisa ditambah lewat UI. Jawab: "sekitar seratus, dan jumlahnya memang tidak tetap karena bisa ditambah dari antarmuka."

A3 "Kalau di tampilan tombolnya sudah disembunyikan, kenapa masih dicek lagi di API? Bukankah itu berlebihan?"

JAWABAN INTI Hafal mati

Karena menyembunyikan tombol bukan pengamanan, itu kenyamanan. Siapa pun bisa memanggil API langsung tanpa lewat tampilan sama sekali — cukup pakai Postman atau `curl` dengan token yang sah. Kalau pengecekannya cuma di tampilan, sistem ini bisa ditembus dalam satu menit. Jadi lapisan tampilan tugasnya membuat antarmuka bersih; lapisan API yang benar-benar menjaga pintu. **Pengecekan di API adalah lapisan yang mengikat.**

AMUNISI

- Pemetaan tampilan dan pemetaan API sengaja disimpan di tabel terpisah. Kelihatannya duplikat, tapi ada gunanya: sebuah tombol bisa disembunyikan dari tampilan tanpa mencabut hak API-nya — misalnya saat fitur sedang diuji internal — dan sebaliknya, endpoint bisa dikunci lebih ketat daripada tampilannya.
- **Sisi API memakai mode ketat.** Kalau ada endpoint dengan resource key yang belum terdaftar sama sekali di tabel pemetaan, permintaannya langsung ditolak. Jadi endpoint baru yang lupa didaftarkan gagal secara mencolok, bukan diam-diam terbuka untuk semua orang. Untuk urusan keamanan, salah arah yang benar adalah menutup, bukan membuka.
- Ada satu penanganan kasus khusus yang layak disebut kalau penguji suka detail: pengecekan permission memakai cache. Kalau cache-nya basi — misalnya seeder dijalankan dari luar kontainer sementara aplikasi jalan di dalam — permission yang sah bisa ikut tertolak dan seluruh aplikasi terkunci sampai cache kedaluwarsa. Jadi sebelum benar-benar menolak, sistem memverifikasi sekali langsung ke tabel di database. Kalau ternyata user memang berhak, cache dibuang dan permintaan diteruskan, sambil mencatat peringatan di log.

JEBAKAN

- Jangan menjawab "supaya lebih aman" saja. Itu jawaban kosong. Sebut skenario konkretnya: **penyerang memanggil API langsung tanpa membuka halaman.**
- Jangan menyebutnya "double validation". Ini bukan validasi ganda atas hal yang sama — dua lapis ini menjaga dua permukaan serangan yang berbeda.

A4 "Kalau sekolah mau menambah jabatan baru dengan hak akses berbeda, apa yang harus diubah?"

JAWABAN INTI Hafal mati

Tidak ada kode yang perlu diubah. Buka halaman Manajemen RBAC, buat peran baru, centang permission yang diinginkan, selesai — berlaku seketika untuk login berikutnya. Kalau ternyata butuh jenis izin yang benar-benar baru, izin itu juga bisa dibuat lewat antarmuka yang sama, lalu diikatkan ke resource key yang relevan. Ini yang saya maksud dengan RBAC dinamis: kebijakan akses adalah data, bukan kode.

AMUNISI

- Empat hal bisa dikelola dari antarmuka: daftar permission, daftar peran beserta centang permission-nya, pemetaan resource key untuk tampilan, dan pemetaan resource key untuk endpoint.
- Permission punya atribut *audience* untuk memisahkan izin milik admin dari izin milik siswa. Tanpa itu, daftar centang di halaman peran akan tercampur dan membingungkan.
- Ada juga jalur permanen lewat enum di backend untuk izin yang memang bawaan sistem dan tidak boleh hilang, lalu disinkronkan lewat seeder. Dua jalur ini hidup berdampingan: enum untuk yang permanen, antarmuka untuk yang situasional.
- **Perubahan berlaku pada login berikutnya**, bukan seketika untuk sesi yang sedang berjalan, karena daftar resource key diambil sekali saat login dan disimpan di session.

JEBAKAN

- Poin terakhir itu **harus kamu sebut sendiri sebelum ditanya**. Kalau penguji yang menemukan bahwa hak akses tidak berubah sampai user login ulang, itu terlihat seperti bug. Kalau kamu yang menyebut duluan sebagai konsekuensi sadar dari menyimpan izin di session demi kecepatan, itu terlihat seperti keputusan desain. Isinya sama, penilaiannya berbeda jauh.

A5 "Yayasan ini punya beberapa cabang. Bagaimana kamu memastikan data cabang satu tidak bocor ke cabang lain?"

JAWABAN INTI Hafal mati

Setiap user terikat pada satu cabang, dan setiap query data selalu difilter berdasarkan cabang milik user itu. Untuk pengguna yang memang berwenang lintas cabang — pengurus yayasan — ada mekanisme perpindahan konteks: frontend mengirim penanda cabang di header permintaan, dan backend hanya menghormatinya kalau user itu punya izin khusus berpindah cabang. Kalau tidak punya izin, header itu diabaikan begitu saja dan datanya tetap cabang asal.

AMUNISI

- Penting: perpindahan cabang itu **hanya berlaku selama satu permintaan**. Konteks cabang diubah di memori, bukan ditulis balik ke data user. Jadi tidak ada kondisi di mana user "tertinggal" di cabang yang salah setelah menutup halaman.
- Izin berpindah cabang sendiri juga sebuah resource key, jadi ia dikelola lewat mekanisme yang sama dengan izin lain — tidak ada daftar pengecualian terpisah yang harus dirawat.

- Pemisahan ini juga sampai ke hal-hal turunan: pengaturan notifikasi, ambang persetujuan otomatis, dan perhitungan saldo semuanya per cabang.

JEBAKAN

- Kalau ditanya "apa yang terjadi kalau ada yang memalsukan header cabang?" — jawabannya bukan "header-nya divalidasi", tapi **header-nya diabaikan tanpa izin**. Perbedaannya penting: sistem tidak menolak permintaan dengan error, ia hanya melanjutkan dengan cabang asal user. Tidak ada bocornya.

Bagian B — Tagihan & Pembayaran

B1 "Bagaimana tagihan seorang siswa terbentuk? Diinput satu-satu?"

JAWABAN INTI Hafal mati

Tidak satu per satu. Admin memilih jenis tagihan, lalu menentukan sasarannya berdasarkan kombinasi jenjang, kelas, dan kategori siswa — misalnya "SPP Agustus untuk seluruh kelas 1 MI kategori reguler". Sistem mencari semua siswa yang cocok, lalu menerbitkan satu tagihan untuk masing-masing dalam satu transaksi database. Nominalnya tidak disimpan di tagihan, melainkan diambil dari jenis tagihannya, jadi satu perubahan tarif tidak perlu menyentuh ribuan baris tagihan.

AMUNISI

- **Periode otomatis.** Kalau tahun ajaran tidak disebut, sistem memakai periode yang sedang aktif di cabang itu. Kalau belum ada periode aktif sama sekali, pembuatan tagihan ditolak dengan pesan yang jelas — bukan dibiarkan menghasilkan tagihan tanpa periode.
- **Anti-tagih-ganda.** Sebelum menerbitkan, sistem memeriksa siswa mana yang sudah punya tagihan jenis ini di periode yang sama, lalu **melewati** mereka. Yang dilewati dilaporkan balik ke admin: berapa yang dibuat, berapa yang dilewati, dan NIS siapa saja. Jadi admin yang tidak sengaja menekan tombol dua kali tidak menciptakan tagihan dobel, dan dia tahu persis apa yang terjadi.
- **Penomoran aman dari tabrakan.** Kode tagihan berformat `TAG-` diikuti tahun-bulan dan nomor urut. Nomor berikutnya dihitung sambil mengunci baris terakhir di dalam transaksi, jadi dua admin yang menyimpan pada detik yang sama tidak bisa mendapat nomor yang sama. Dan karena kode itu sendiri kunci utama tabel, database menolak duplikat sebagai lapisan pengaman terakhir.
- Setiap tagihan yang terbit memicu peristiwa yang menjalankan pengiriman email pemberitahuan.

JEBAKAN

- Kalau ditanya kenapa nominal tidak disimpan di tagihan: jawab dari sisi konsekuensi, bukan teori normalisasi. "Kalau tarif SPP naik, saya cukup mengubah satu baris jenis tagihan." Itu jawaban yang terasa nyata.

B2 "Kenapa tagihan dihubungkan ke siswa lewat NIS, bukan lewat ID siswa?"

JAWABAN INTI Hafal mati

Karena NIS adalah identitas yang dipakai sekolah di dunia nyata — tertulis di kwitansi, disebut orang tua saat bertanya, dipakai di berkas administrasi. Menyambungkan lewat NIS membuat data sistem cocok dengan dokumen fisik tanpa penerjemahan. Konsekuensinya saya sadari: NIS jadi tidak boleh diubah sembarangan, karena mengubahnya memutus kaitan ke seluruh riwayat tagihan siswa itu.

AMUNISI

- Ini termasuk hal yang saya catat sebagai keterbatasan yang diketahui, bukan yang tidak terpikirkan. Perbaikan yang benar adalah menyambung lewat ID internal dan menjadikan NIS sekadar atribut yang bisa berubah.
- Untuk saat ini risikonya ditekan dari sisi prosedur: perubahan NIS bukan operasi rutin dan berada di balik izin perubahan data siswa.

JEBAKAN

- **Jangan berpura-pura ini rancangan sempurna.** Penguji yang paham basis data akan langsung melihat kelemahannya. Mengakui duluan, menjelaskan alasan praktisnya, dan menyebutkan perbaikan yang tepat — itu jauh lebih kuat daripada bertahan.
- Jangan sebut ini "bug". Ini **keputusan desain dengan konsekuensi yang diketahui**. Dua hal yang sangat berbeda di mata penguji.

B3 "Apa bedanya pembayaran tunai dan pembayaran daring di sistem kamu?"

JAWABAN INTI Hafal mati

Beda titik masuknya. Tunai dicatat admin di kantor: admin membuka tagihan, memasukkan nominal dan nama pembayar, sistem langsung mencatat pembayaran dan memperbarui sisa tagihan. Daring dimulai dari siswa di portal, uangnya lewat Midtrans, dan sistem baru mencatat pembayaran setelah menerima konfirmasi resmi dari Midtrans. Yang sama: keduanya berujung pada satu jenis catatan pembayaran, memperbarui akumulasi terbayar pada tagihan yang sama, dan memicu pengiriman kwitansi lewat email.

AMUNISI

- **Cicilan didukung di kedua jalur.** Tagihan menyimpan akumulasi terbayar, dan status berubah menjadi lunas hanya ketika akumulasinya mencapai nominal penuh. Jadi orang tua bisa membayar SPP tiga kali dalam sebulan.
- **Pembayaran berlebih ditolak.** Sebelum mencatat, sistem menjumlahkan yang sudah terbayar dengan yang sedang dibayarkan; kalau melebihi nominal tagihan, permintaannya ditolak. Tagihan yang sudah lunas juga tidak bisa dibayar lagi.
- Ada pula pelunasan massal untuk kasus admin menerima pembayaran beberapa tagihan sekaligus dari satu orang tua.
- Kode pembayaran punya penomoran sendiri, terpisah dari kode tagihan.

JEBAKAN

- Kalau ditanya "kenapa tidak semua dipaksa daring saja?" — jawab dari realitas lapangan, bukan teknis: sebagian besar orang tua di lingkungan sekolah ini masih membayar tunai di kantor, dan mematikan jalur itu berarti sistemnya tidak bisa dipakai. Pembayaran daring itu tambahan, bukan pengganti.

B4 "Bagaimana sistem kamu tahu bahwa pembayaran daring benar-benar berhasil?"

JAWABAN INTI Hafal mati

Bukan dari pengembalian halaman setelah siswa membayar. Halaman itu hanya tampilan, dan siswa bisa saja menutup peramban sebelum sampai ke sana. Yang menentukan adalah **webhook**: Midtrans memanggil endpoint di server kami dari server mereka sendiri, membawa status resmi transaksi. Status pembayaran di sistem hanya berubah karena pemanggilan itu. Jadi keputusan "lunas atau belum" selalu diambil di sisi server berdasarkan sumber yang berwenang, tidak pernah berdasarkan apa yang dilaporkan peramban siswa.

AMUNISI

- **Kalau webhook tidak sampai** — misalnya jaringan bermasalah — ada jaring pengaman: admin bisa menekan tombol sinkronisasi yang menarik status langsung dari API Midtrans. Yang penting, hasil tarikan itu diproses lewat **jalur pemrosesan yang persis sama** dengan webhook. Tidak ada logika kedua yang bisa menyimpang dari yang pertama.
- **Satu tagihan tidak bisa punya dua transaksi hidup**. Saat siswa menekan bayar untuk tagihan yang sudah punya transaksi tertunda, sistem tidak membuat transaksi baru — ia mengembalikan transaksi yang lama beserta tokennya, jadi siswa dilanjutkan ke halaman pembayaran yang sama.
- **Kepemilikan diverifikasi**. Siswa hanya bisa membayar tagihan atas namanya sendiri; NIS pemilik tagihan dicocokkan dengan NIS pengguna yang login.
- Transaksi juga punya masa berlaku — bawaannya 24 jam — supaya transaksi tertunda tidak menggantung selamanya dan mengunci tagihan.

JEBAKAN

- Kalimat "**diperiksa di sisi server, bukan di sisi peramban**" adalah inti jawaban ini. Kalau cuma satu kalimat yang sempat keluar, itu kalimatnya.
- Jangan pernah bilang "kalau halaman sukses muncul, berarti lunas". Itu justru kesalahan klasik yang pertanyaan ini incar.

B5 "Endpoint webhook itu terbuka untuk publik tanpa autentikasi. Bukankah itu lubang keamanan?"

JAWABAN INTI Hafal mati

Memang terbuka, dan memang harus. Yang memanggilnya server Midtrans, bukan pengguna yang login — tidak ada token yang bisa mereka kirim. Pengamanannya bukan autentikasi, melainkan **tanda tangan kriptografis**. Setiap notifikasi membawa sidik SHA-512 yang dihitung dari nomor pesanan, kode status, nominal, dan kunci rahasia server kami. Kami menghitung ulang sidik itu dan membandingkannya. Kunci rahasia tidak pernah keluar dari server, jadi pihak lain tidak bisa membuat sidik yang cocok.

AMUNISI

- Perbandingan sidiknya memakai fungsi **waktu-tetap**, bukan perbandingan string biasa. Perbandingan biasa berhenti di karakter pertama yang berbeda, dan selisih waktunya — sekecil apa pun — bisa dipakai menebak sidik yang benar karakter demi karakter. Fungsi waktu-tetap selalu memakan waktu sama.
- Tanda tangan bukan satu-satunya lapis. Nominal yang dikirim juga dicocokkan ulang dengan yang kami simpan sendiri; kalau berbeda, ditolak. Jadi bahkan notifikasi bertanda tangan sah pun tidak bisa mengubah nominal.
- **Semua notifikasi dicatat mentah-mentah sebelum divalidasi**. Notifikasi yang ditolak pun meninggalkan jejak. Ini penting untuk penelusuran: kalau ada yang mencoba menembus, buktinya ada.

JEBAKAN

- Ini pertanyaan jebakan yang menguji apakah kamu paham bedanya **autentikasi** dan **verifikasi keaslian pesan**. Jangan terpancing menjanjikan akan "menambahkan autentikasi" — itu justru jawaban yang salah dan akan merusak integritasnya.
- Jangan sebut kunci server sebagai "password". Sebut **server key**, dan tegaskan ia tidak pernah dikirim ke mana pun.

B6 "Bagaimana kalau notifikasi pembayaran datang dua kali, atau dua notifikasi masuk bersamaan?"

JAWABAN INTI Hafal mati

Ada penjagaan berlapis. Kalau status yang dikirim sama dengan status yang sudah tersimpan, sistem tidak melakukan apa-apa dan menjawab sukses. Perpindahan status juga dibatasi aturan — dari status akhir seperti kedaluwarsa atau ditolak tidak bisa berpindah ke mana pun. Dan sebelum mencatat pembayaran, sistem memastikan belum ada pembayaran dengan nomor pesanan yang sama. Untuk dua notifikasi yang masuk bersamaan, seluruh pemrosesan berjalan di dalam transaksi database dengan penguncian baris, jadi yang kedua menunggu yang pertama selesai lalu melihat status yang sudah diperbarui.

AMUNISI

- Istilah untuk sifat ini: **idempoten** — pemanggilan berulang menghasilkan keadaan akhir yang sama dengan pemanggilan sekali. Ini kebutuhan wajib untuk webhook, karena penyedia pembayaran mana pun akan mengulang pengiriman jika belum menerima balasan sukses.

- Penguncian barisnya juga mencoba ulang bila terjadi kebuntuan antar transaksi.
- Pembayaran berlebih dicegah **dua kali**: saat siswa memulai transaksi, dan sekali lagi saat webhook memproses. Alasannya nyata: di antara dua momen itu, admin bisa saja mencatat pembayaran tunai untuk tagihan yang sama.

JEBAKAN

- Kata **idempoten** sangat berbobot di sini, tapi hanya kalau kamu bisa menjelaskan artinya begitu ditanya balik. Kalau ragu, pakai kalimat biasa: "notifikasi kedua tidak menghasilkan pembayaran kedua."

Bagian C — Persetujuan Pengeluaran

c1 "Coba jelaskan alur pengajuan pengeluaran dari awal sampai uangnya cair."

JAWABAN INTI Hafal mati

Empat tahap. Pengaju membuat draf, lalu mengajukannya. Penyetuju memeriksa dan memilih menyetujui atau menolak. Kalau disetujui, tahap terakhir adalah pencairan — dan **baru pada saat pencairan itulah catatan pengeluaran yang sesungguhnya terbentuk**, yang masuk ke laporan keuangan. Jadi pengajuan yang disetujui tapi belum dicairkan belum mempengaruhi laporan. Setiap perpindahan status dicatat di log persetujuan lengkap dengan siapa pelakunya, kapan, dan catatannya.

AMUNISI

- **Pengajuan yang ditolak tidak mati.** Ia bisa diperbaiki dan diajukan ulang. Yang bisa disunting hanya yang berstatus draf atau ditolak — begitu diajukan, isinya terkunci supaya penyetuju tidak menilai angka yang berbeda dari yang akhirnya dicairkan.
- **Penolakan wajib beralasan.** Sistem menolak penolakan tanpa alasan. Alasannya masuk ke log dan dikirimkan ke pengaju lewat email, jadi ia tahu apa yang harus diperbaiki.
- **Periode pembukuan ditentukan saat pencairan**, bukan saat pengajuan. Pengajuan yang diajukan di akhir periode lama tapi dicairkan di periode baru akan tercatat di periode baru — sesuai kapan uangnya benar-benar keluar.
- Setiap perpindahan status dibungkus transaksi database, jadi tidak ada kondisi di mana status berubah tapi lognya gagal tercatat.

JEBAKAN

- **Jangan bilang "berjenjang" atau "multi-level".** Alurnya satu tingkat: satu penyetuju, satu keputusan. Kalau kamu menyebut berjenjang lalu penguji minta tunjukkan tingkat keduanya, kamu tidak punya apa-apa untuk ditunjukkan. Kalau memang ditanya soal berjenjang, jawab jujur: "Untuk saat ini satu tingkat. Struktur lognya sudah menampung riwayat perpindahan status, jadi menambah tingkat berikutnya tidak perlu merombak model datanya."

C2 "Apa yang mencegah sekolah mengeluarkan uang yang tidak dimilikinya?"

JAWABAN INTI Hafal mati

Ada penjagaan saldo yang dijalankan dua kali: saat pengajuan diajukan, dan sekali lagi saat akan dicairkan. Saldo tersedia dihitung sebagai total pemasukan dikurangi pengeluaran yang sudah cair, lalu **dikurangi lagi dengan seluruh pengajuan yang masih berjalan** — yang sudah diajukan atau sudah disetujui tapi belum cair. Jadi uang yang sudah "dijatah" untuk pengajuan lain tidak bisa dijatah dua kali. Kalau nominalnya melebihi saldo tersedia, sistem menolak dengan pesan yang menyebutkan angka saldo dan angka yang dibutuhkan.

AMUNISI

- **Perhitungannya selalu satu cabang penuh, tidak pernah difilter per periode.** Ini keputusan sadar: kalau saldo dihitung per tahun ajaran, seseorang bisa mengajukan pengeluaran di periode yang saldonya "kelihatan longgar" padahal cabangnya sendiri sudah defisit. Menghitung branch-wide menutup celah itu.
- **Dicek dua kali karena keadaan bisa berubah di antaranya.** Antara pengajuan disetujui dan dicairkan, bisa saja ada pengeluaran lain yang cair duluan. Pengecekan pertama tidak menjamin apa pun untuk momen kedua.
- Perhitungan yang sama dipakai untuk menampilkan ringkasan saldo di halaman pengeluaran dan dasbor — satu sumber, jadi angka yang dilihat admin persis angka yang dipakai sistem untuk menolak.

JEBAKAN

- Kalau ditanya "bagaimana kalau dua orang mengajukan bersamaan?" — jawab apa adanya: perhitungannya dilakukan di dalam transaksi database, tapi **ini termasuk bagian yang paling sulit dijamin sepenuhnya** pada beban tinggi. Untuk skala satu yayasan dengan beberapa pengaju, risikonya kecil. Mengakui batas ketelitian jauh lebih baik daripada mengklaim kekebalan yang tidak bisa kamu buktikan kalau diminta demonstrasi.

C3 "Apa itu persetujuan otomatis, dan kapan berlaku?"

JAWABAN INTI Hafal mati

Tiap cabang bisa menetapkan ambang nominal. Pengajuan di bawah ambang itu langsung disetujui begitu diajukan, tanpa menunggu penyetuju. Alasannya praktis: kepala yayasan tidak perlu diganggu untuk pembelian spidol seharga lima puluh ribu. Fiturnya harus dinyalakan secara sengaja per cabang dan ambangnya harus lebih dari nol — jadi kalau tidak diatur sama sekali, semua pengajuan tetap lewat persetujuan manual.

AMUNISI

- Persetujuan otomatis **tetap mencatat log** dengan keterangan bahwa ini disetujui otomatis karena berada dalam batas ambang. Tidak ada pengajuan yang berpindah status tanpa jejak.
- Persetujuan otomatis **tidak melewati pengecekan saldo**. Nominal kecil pun tetap ditolak kalau saldo cabang tidak cukup, karena penjagaan saldo dijalankan lebih dulu, sebelum jalur persetujuan dipilih.
- Yang dilewati hanya tahap persetujuan. **Pencairan tetap manual** — masih butuh seseorang menekan tombol cairkan. Jadi tidak ada uang yang bergerak tanpa tindakan manusia.

JEBAKAN

- Poin terakhir itu penting dan sering ditanya sebagai "berarti uangnya bisa keluar sendiri?" Jawabannya tegas: **tidak**. Yang otomatis persetujuannya, bukan pencairannya.

C4 "Bagaimana kamu bisa membuktikan siapa yang menyetujui pengeluaran tertentu?"

JAWABAN INTI Hafal mati

Ada tabel log persetujuan yang mencatat setiap perpindahan status: dari status apa ke status apa, oleh siapa, kapan, dan catatan yang ditulis. Yang dicatat bukan keadaan akhirnya saja, tapi seluruh riwayat perpindahannya. Jadi satu pengajuan yang pernah ditolak lalu diperbaiki dan disetujui punya jejak lengkap kedua peristiwa itu, bukan cuma hasil akhirnya.

AMUNISI

- Log ditulis **di dalam transaksi yang sama** dengan perubahan status. Tidak ada kemungkinan status berubah tapi lognya hilang, atau sebaliknya.
- Persetujuan otomatis juga masuk log, dicatat atas nama pengaju dengan keterangan bahwa penyetujuannya otomatis — bukan dibiarkan kosong seolah tidak ada yang bertanggung jawab.
- Ini yang membuat catatan pengeluaran bisa ditelusuri balik: dari baris pengeluaran di laporan, ada rujukan ke pengajuan asalnya, dan dari sana ke seluruh riwayat persetujuannya.

JEBAKAN

- Kalau ditanya "apakah log bisa diubah?" — jawab jujur: **secara teknis bisa**, karena ini baris di database biasa dan tidak ada mekanisme anti-rusak seperti rantai hash. Yang melindunginya adalah tidak adanya antarmuka apa pun untuk menyunting atau menghapusnya. Menjanjikan sesuatu yang tidak diimplementasikan adalah cara tercepat kehilangan kepercayaan penguji.

Bagian D — Notifikasi & Kenaikan Kelas

D1 "Bagaimana sebenarnya cara kerja notifikasi email di sistem kamu?"

JAWABAN INTI Hafal mati

Ada enam jenis email, dipicu dua cara: sebagian oleh peristiwa — tagihan terbit, pembayaran tercatat, pengajuan berpindah status — dan sebagian terjadwal, yaitu pengingat jatuh tempo dan pemberitahuan keterlambatan yang berjalan tiap pagi. Yang membedakan dari sekadar "kirim email": sebelum satu email keluar, ia harus melewati enam gerbang berurutan, dan **hasil setiap kemungkinan tercatat di log notifikasi** — termasuk yang tidak jadi dikirim, lengkap dengan alasannya.

AMUNISI enam gerbangnya, berurutan

1. **Jenis notifikasi ini aktif untuk cabang ini?** Tiap cabang punya sakelar sendiri. Tidak aktif, dilewati dengan alasan "dinonaktifkan".

2. **Ada alamat tujuannya?** Sistem mencari berurutan: email akun siswa, lalu wali, lalu ibu, lalu ayah. Tidak ada satu pun, dilewati dengan alasan "tidak ada email".
3. **Alamat itu sudah berhenti berlangganan?** Dicek per jenis notifikasi, jadi seseorang bisa berhenti menerima pengingat tapi tetap menerima kwitansi.
4. **Formatnya sah?** Alamat yang salah ketik dilewati, bukan dicoba lalu gagal.
5. **Melebihi batas kirim?** Maksimal seratus email per jam per cabang. Ini jaring pengaman terhadap perulangan tak terduga.
6. **Kirim.**

AMUNISI tambahan

- Kenapa selalu dicatat: **email adalah efek yang tidak bisa dibatalkan**. Kalau ada orang tua bertanya "saya tidak menerima pemberitahuan", admin harus bisa menjawab dengan bukti — terkirim, dilewati, atau gagal, dan kenapa. Tanpa log, pertanyaan itu tidak terjawab.
- Pengaturannya per cabang karena tiap cabang punya kebiasaan administrasi berbeda; termasuk berapa hari sebelum jatuh tempo pengingat dikirim.

JEBAKAN

- Jangan bilang "emailnya pasti sampai". Sistem menjamin **percobaan pengiriman tercatat**, bukan keterkirim di kotak masuk. Yang terakhir di luar kendali aplikasi mana pun.

D2 "Kenapa email tidak langsung dikirim saat tombolnya ditekan?"

JAWABAN INTI Hafal mati

Karena mengirim email butuh waktu dan bisa gagal. Kalau dikirim langsung dalam siklus permintaan, admin yang menyimpan seratus tagihan harus menunggu seratus pengiriman selesai — dan kalau server email sedang mati, penyimpanan tagihannya ikut gagal padahal datanya sendiri tidak bermasalah. Jadi email dimasukkan **antrian**: permintaan admin selesai seketika, dan proses pekerja terpisah yang mengurus pengirimannya di belakang layar.

AMUNISI

- **Percobaan ulang otomatis** sampai tiga kali, dengan jeda bertambah: sepuluh detik, tiga puluh detik, enam puluh detik. Gangguan sesaat pada server email sembuh sendiri tanpa siapa pun tahu.
- Kalau tiga percobaan habis, log ditandai gagal beserta pesan kesalahannya, dan admin bisa memilih baris-baris gagal itu lalu menjalankan pengiriman ulang secara massal dari antarmuka.
- Karena itu, status "terkirim" di log sebenarnya berarti **"berhasil masuk antrian"**, dan akan dikoreksi menjadi gagal kalau pengirimannya benar-benar gagal.
- Notifikasi memakai antrian terpisah dari pekerjaan berat seperti impor dan ekspor, supaya email tidak mengantre di belakang berkas Excel besar.

JEBAKAN

- Kalau nanti demo dan email tidak muncul, penyebab paling mungkin adalah **pekerja antriannya tidak berjalan**, atau berjalan tapi tidak memproses antrian notifikasi. Ini gagal dalam diam — tidak ada pesan kesalahan yang muncul, emailnya hanya tidak keluar. Pastikan pekerja antrian menyala sebelum masuk ruang sidang.

D3 "Bagaimana kamu mencegah orang tua menerima email yang sama berkali-kali?"

JAWABAN INTI Hafal mati

Untuk email terjadwal ada tabel catatan pengiriman dengan kunci gabungan: kode tagihan, jenis notifikasi, dan tanggal. Sebelum mengirim, sistem memeriksa apakah kombinasi itu sudah ada. Jadi kalau penjadwal kebetulan berjalan dua kali dalam sehari, atau ada yang menjalankan perintahnya manual, pengingat untuk tagihan yang sama tidak terkirim dua kali di hari yang sama. Untuk email yang dipicu peristiwa, pemicunya memang sekali per kejadian. Di atas keduanya ada pembatas seratus email per jam per cabang sebagai jaring terakhir.

AMUNISI

- Kuncinya menyertakan **tanggal**, bukan cuma tagihan dan jenis. Ini disengaja: pengingat jatuh tempo memang harus bisa terkirim lagi di hari berikutnya. Yang dicegah adalah dobel di hari yang sama, bukan pengiriman berulang antar hari.
- Bawaannya pengingat dikirim pada tujuh hari, tiga hari, dan tepat di hari jatuh tempo — dan itu bisa diubah per cabang dari antarmuka.

JEBAKAN

- Kalau ditanya "kenapa tidak pakai kunci unik di database saja?" — jawaban jujurnya: pengecekan dilakukan di lapisan aplikasi, dan pada beban tinggi secara teori masih ada celah balapan. Untuk volume sistem ini — puluhan email per hari per cabang — risikonya dapat diabaikan. Jangan mengklaim jaminan tingkat database kalau yang ada pengecekan tingkat aplikasi.

D4 "Bagaimana verifikasi email dengan OTP bekerja?"

JAWABAN INTI Hafal mati

Pengguna meminta verifikasi, sistem membuat kode enam digit acak, menyimpannya di cache dengan masa berlaku sepuluh menit, lalu mengirimkannya ke alamat yang diverifikasi. Saat kode dimasukkan, sistem membandingkannya dengan yang tersimpan. Cocok, alamat ditandai terverifikasi dan kodenya langsung dihapus dari cache — jadi satu kode hanya bisa dipakai sekali. Untuk mencegah penyalahgunaan, permintaan kode dibatasi tiga kali per sepuluh menit per pengguna.

AMUNISI

- Kodenya dibuat dengan pembangkit acak yang aman secara kriptografis, bukan fungsi acak biasa.
- **Kode disimpan di cache, bukan di tabel permanen.** Konsekuensi bagus: kode kedaluwarsa hilang sendiri tanpa perlu pembersihan berkala. Konsekuensi yang harus diakui: kalau cache dikosongkan, kode yang beredar jadi tidak berlaku dan pengguna harus meminta ulang.
- Ada jalur terpisah untuk memverifikasi email ayah, ibu, atau wali — bukan hanya email akun siswa. Ini penting karena penerima notifikasi tagihan sering kali orang tua, bukan siswanya sendiri.
- **Atur ulang kata sandi memakai mekanisme berbeda:** bukan kode enam digit, melainkan token acak enam puluh empat karakter dengan masa berlaku satu jam, dikirim sebagai tautan. Kodenya sengaja dibedakan karena taruhannya berbeda — kode enam digit terlalu mudah ditebak untuk sesuatu yang bisa mengambil alih akun.

JEBAKAN

- Perbedaan terakhir itu **sebutkan sendiri**. Penguji yang jeli akan bertanya "kenapa reset password tidak pakai OTP enam digit juga?" — dan kalau kamu sudah menjelaskannya duluan, kamu terlihat memikirkan model ancaman, bukan sekadar menyalin pola.

D5 "Kenaikan kelas satu angkatan sekaligus — bagaimana kalau ada yang salah?"

JAWABAN INTI Hafal mati

Ada empat operasi: naik kelas, tinggal kelas, lulus, dan pindah jenjang. Semuanya dijalankan sebagai satu kesatuan transaksi database — kalau ada satu siswa yang gagal diproses, seluruhnya dibatalkan, tidak ada kondisi setengah jalan. Di atas itu, setiap operasi menghasilkan satu **batch** dengan nomor sendiri, dan tiap siswa yang terpengaruh tercatat terikat ke batch itu. Jadi kalau ternyata salah pilih, ada tombol **batalkan batch** yang mengembalikan seluruh siswa ke kondisi sebelumnya.

AMUNISI

- **Pembatalan tidak bisa dilakukan dua kali.** Batch yang sudah dibatalkan ditandai, dan percobaan membatalkannya lagi ditolak — supaya tidak ada yang "membatalkan pembatalan" dan mengacak data.
- **Perpindahan jenjang divalidasi.** Tidak semua perpindahan masuk akal — misalnya KB langsung ke MI. Ada pemeriksaan yang menolak perpindahan yang tidak diizinkan.
- **Kelas tujuan bisa dipilih manual.** Awalnya kelas berikutnya ditentukan otomatis dari urutan tingkat, tapi itu tidak cukup untuk sekolah yang punya beberapa kelas sejajar di satu tingkat — misalnya 1A dan 1B. Jadi sekarang admin bisa menentukan kelas tujuan secara eksplisit, dan penentuan otomatis dipakai sebagai bawaan kalau tidak ditentukan.
- Kenaikan kelas selalu terikat pada tahun ajaran tujuan, jadi riwayat kelas seorang siswa per periode tetap utuh dan tidak tertimpa.

JEBAKAN

- Fitur ini paling mungkin diminta didemokan langsung karena akibatnya kelihatan. **Kalau mendemokan, siapkan batch percobaan yang memang untuk dibatalkan**, jangan menyentuh data yang kamu pakai untuk demo lain — dan jangan lupa membatalkannya sebelum lanjut ke bagian berikutnya.
- Jangan menyebutnya "undo" dalam arti bisa berkali-kali mundur. Yang ada adalah **pembatalan satu batch, sekali**.

Ringkasan — kalimat jangkar

Kalau semua hilang dari kepala, tarik satu dari sini. Tiap kalimat cukup untuk membuka jawaban dan memberi waktu berpikir.

1. **Login** — password di-hash bcrypt, yang keluar token Sanctum berlaku 8 jam; backend-nya stateless.
2. **Hak akses** — kode tidak pernah menyebut nama permission, hanya *resource key*; pemetaannya data di database, bukan kode.
3. **Dua lapis** — menyembunyikan tombol itu kenyamanan; **pengecekan di API yang mengikat**.

4. **Peran baru** — tidak ada kode yang berubah, cukup lewat antarmuka; berlaku pada login berikutnya.
5. **Antar cabang** — header cabang hanya dihormati kalau user punya izin berpindah; kalau tidak, diabaikan diam-diam.
6. **Tagihan** — diterbitkan massal per jenjang-kelas-kategori; yang sudah punya dilewati dan dilaporkan.
7. **NIS** — disambung lewat NIS karena itu identitas dunia nyata; konsekuensinya saya sadari.
8. **Tunai vs daring** — beda titik masuk, sama muaranya; keduanya mendukung cicilan.
9. **Lunas** — **diputuskan di sisi server lewat webhook**, tidak pernah dari halaman peramban.
10. **Webhook terbuka** — dijaga tanda tangan SHA-512, dibandingkan waktu-tetap; kunci server tak pernah keluar.
11. **Notifikasi ganda** — idempoten: pemanggilan kedua tidak menghasilkan pembayaran kedua.
12. **Pengeluaran** — draf, ajukan, setuju, cairkan; catatan keuangan lahir **saat pencairan**, bukan saat persetujuan.
13. **Saldo** — dicek dua kali, dihitung satu cabang penuh, sudah memotong pengajuan yang masih berjalan.
14. **Otomatis** — yang otomatis persetujuannya, **bukan pencairannya**.
15. **Jejak** — log mencatat seluruh riwayat perpindahan status, bukan hasil akhirnya saja.
16. **Email** — enam gerbang sebelum terkirim, dan **setiap kemungkinan meninggalkan satu baris log**.
17. **Antrian** — email tidak dikirim dalam siklus permintaan; tiga kali coba ulang dengan jeda bertambah.
18. **Anti-dobel** — kunci tagihan + jenis + tanggal, jadi sehari sekali, tapi tetap bisa besok.
19. **OTP** — enam digit, sepuluh menit, sekali pakai; reset kata sandi pakai token panjang karena taruhannya lebih besar.
20. **Kenaikan kelas** — satu transaksi utuh, dikelompokkan per batch, dan batch bisa dibatalkan sekali.

Pola yang menyatukan semuanya

Kalau penguji bertanya sesuatu yang tidak ada di daftar ini, **kembalikan ke salah satu dari empat pola berikut** — hampir semua keputusan teknis di sistem ini turunan dari sini:

- **Keputusan penting diambil di sisi server.** Tampilan hanya mencerminkan, tidak menentukan. Berlaku untuk hak akses, status lunas, dan penjagaan saldo.
- **Kebijakan adalah data, bukan kode.** Permission, ambang persetujuan otomatis, pengaturan notifikasi — semuanya bisa diubah tanpa deploy ulang.
- **Setiap kemungkinan meninggalkan jejak.** Termasuk yang gagal dan yang dilewati. Sistem yang gagal dalam diam tidak bisa diaudit.
- **Operasi yang menyentuh banyak baris dibungkus transaksi.** Tidak ada keadaan setengah jadi — berhasil semua atau batal semua.

Yang harus diakui duluan, sebelum ditemukan penguji

Tiga hal ini adalah keterbatasan nyata. **Sebut sendiri di momen yang tepat**, jangan tunggu digali:

- Tagihan disambung ke siswa lewat NIS, bukan ID internal. Mengubah NIS memutus kaitan riwayat.
- Alur persetujuan satu tingkat, belum berjenjang. Struktur lognya sudah siap menampung, tapi tingkat keduanya belum ada.
- Pengecekan saldo dan anti-dobel email dilakukan di lapisan aplikasi. Untuk skala satu yayasan cukup; untuk beban tinggi, penjaminannya perlu naik ke tingkat database.

Mengakui batas itu bukan kelemahan. **Yang dinilai penguji bukan sistem tanpa cacat — itu tidak ada — tapi apakah kamu tahu di mana cacatnya.**